```sql
USE HR_DATABASE_MANAGEMENT;

CREATE TABLE regions ( region_id INT PRIMARY KEY, region_name VARCHAR ( 25) DEFAULT NULL );
CREATE TABLE countries ( country_id CHAR (2) PRIMARY KEY, country_name VARCHAR (40) DEFAULT NULL, region_id INT NOT NULL, FOREIGN KEY (region_id) REFERENCES regions (region_id) ON DELETE CASCADE ON UPDATE CASCADE );
CREATE TABLE locations ( location_id INT PRIMARY KEY, street_address VARCHAR (40) DEFAULT NULL, postal_code VARCHAR (12) DEFAULT NULL, city VARCHAR (30) NOT NULL, state_province VARCHAR (25) DEFAULT NULL, country_id CHAR (2) NOT NULL, FOREIGN KEY (country_id) REFERENCES countries (country_id) ON DELETE CASCADE ON UPDATE CASCADE );
CREATE TABLE jobs ( job_id INT PRIMARY KEY, job_title VARCHAR (35) NOT NULL, min_salary DECIMAL (8, 2) DEFAULT NULL, max_salary DECIMAL (8, 2) DEFAULT NULL );
CREATE TABLE departments ( department_id INT PRIMARY KEY, department_name VARCHAR (30) NOT NULL, location_id INT DEFAULT NULL, FOREIGN KEY (location_id) REFERENCES locations (location_id) ON DELETE CASCADE ON UPDATE CASCADE );
CREATE TABLE employees ( employee_id INT PRIMARY KEY, first_name VARCHAR (20) DEFAULT NULL, last_name VARCHAR (25) NOT NULL, email VARCHAR (100) NOT NULL, phone_number VARCHAR (20) DEFAULT NULL, hire_date DATE NOT NULL, job_id INT NOT NULL, salary DECIMAL (8, 2) NOT NULL, manager_id INT DEFAULT NULL, department_id INT DEFAULT NULL, FOREIGN KEY (job_id) REFERENCES jobs (job_id) ON DELETE CASCADE ON UPDATE CASCADE, FOREIGN KEY (department_id) REFERENCES departments (department_id) ON DELETE CASCADE ON UPDATE CASCADE, FOREIGN KEY (manager_id) REFERENCES employees (employee_id) );
CREATE TABLE dependents ( dependent_id INT PRIMARY KEY, first_name VARCHAR (50) NOT NULL, last_name VARCHAR (50) NOT NULL, relationship VARCHAR (25) NOT NULL, employee_id INT NOT NULL, FOREIGN KEY (employee_id) REFERENCES employees (employee_id) ON DELETE CASCADE ON UPDATE CASCADE );

/*Data for the table regions */
INSERT INTO regions(region_id,region_name) VALUES (1,'Europe');
INSERT INTO regions(region_id,region_name) VALUES (2,'Americas');
INSERT INTO regions(region_id,region_name) VALUES (3,'Asia');
INSERT INTO regions(region_id,region_name) VALUES (4,'Middle East and Africa');

/*Data for the table countries */
INSERT INTO countries(country_id,country_name,region_id) VALUES ('AR','Argentina',2);
INSERT INTO countries(country_id,country_name,region_id) VALUES ('AU','Australia',3);
INSERT INTO countries(country_id,country_name,region_id) VALUES ('BE','Belgium',1);
```

```
21  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('BR','Brazil',2);
22  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('CA','Canada',2);
23  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('CH','Switzerland',1);
24  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('CN','China',3);
25  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('DE','Germany',1);
26  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('DK','Denmark',1);
27  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('EG','Egypt',4);
28  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('FR','France',1);
29  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('HK','HongKong',3);
30  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('IL','Israel',4);
31  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('IN','India',3);
32  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('IT','Italy',1);
33  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('JP','Japan',3);
34  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('KW','Kuwait',4);
35  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('MX','Mexico',2);
36  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('NG','Nigeria',4);
37  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('NL','Netherlands',1);
38  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('SG','Singapore',3);
39  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('UK','United Kingdom',1);
40  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('US','United States of America',2);
41  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('ZM','Zambia',4);
42  INSERT INTO countries(country_id,country_name,region_id) VALUES
      ('ZW','Zimbabwe',4);
43
44  /*Data for the table locations */
45  INSERT INTO locations
      (location_id,street_address,postal_code,city,state_province,country_id)
      VALUES (1400,'2014 Jabberwocky Rd','26192','Southlake','Texas','US');
```

```
46 INSERT INTO locations
     (location_id,street_address,postal_code,city,state_province,country_id)
     VALUES (1500,'2011 Interiors Blvd','99236','South San
     Francisco','California','US');
47 INSERT INTO locations
     (location_id,street_address,postal_code,city,state_province,country_id)
     VALUES (1700,'2004 Charade Rd','98199','Seattle','Washington','US');
48 INSERT INTO locations
     (location_id,street_address,postal_code,city,state_province,country_id)
     VALUES (1800,'147 Spadina Ave','M5V 2L7','Toronto','Ontario','CA');
49 INSERT INTO locations
     (location_id,street_address,postal_code,city,state_province,country_id)
     VALUES (2400,'8204 Arthur St',NULL,'London',NULL,'UK');
50 INSERT INTO locations
     (location_id,street_address,postal_code,city,state_province,country_id)
     VALUES (2500,'Magdalen Centre, The Oxford Science Park','OX9
     9ZB','Oxford','Oxford','UK');
51 INSERT INTO locations
     (location_id,street_address,postal_code,city,state_province,country_id)
     VALUES (2700,'Schwanthalerstr. 7031','80925','Munich','Bavaria','DE');
52
53 /*Data for the table jobs */
54 INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES (1,'
     Public Accountant',4200.00,9000.00);
55 INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES
     (2,'Accounting Manager',8200.00,16000.00);
56 INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES
     (3,'Administration Assistant',3000.00,6000.00);
57 INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES
     (4,'President',20000.00,40000.00);
58 INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES
     (5,'Administration Vice President',15000.00,30000.00);
59 INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES
     (6,'Accountant',4200.00,9000.00);
60 INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES
     (7,'Finance Manager',8200.00,16000.00);
61 INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES (8,'Human
     Resources Representative',4000.00,9000.00);
62 INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES
     (9,'Programmer',4000.00,10000.00);
63 INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES
     (10,'Marketing Manager',9000.00,15000.00);
64 INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES
     (11,'Marketing Representative',4000.00,9000.00);
65 INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES
     (12,'Public Relations Representative',4500.00,10500.00);
66 INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES
     (13,'Purchasing Clerk',2500.00,5500.00);
67 INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES
```

```
    (14,'Purchasing Manager',8000.00,15000.00);
68  INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES (15,'
    Sales Manager',10000.00,20000.00);
69  INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES (16,'
    Sales Representative',6000.00,12000.00);
70  INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES
    (17,'Shipping Clerk',2500.00,5500.00);
71  INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES (18,'
    Stock Clerk',2000.00,5000.00);
72  INSERT INTO jobs(job_id,job_title,min_salary,max_salary) VALUES (19,'
    Stock Manager',5500.00,8500.00);
73
74  /*Data for the table departments */
75  INSERT INTO departments(department_id,department_name,location_id) VALUES
    (1,'Administration',1700);
76  INSERT INTO departments(department_id,department_name,location_id) VALUES
    (2,'Marketing',1800);
77  INSERT INTO departments(department_id,department_name,location_id) VALUES
    (3,'Purchasing',1700);
78  INSERT INTO departments(department_id,department_name,location_id) VALUES
    (4,'Human Resources',2400);
79  INSERT INTO departments(department_id,department_name,location_id) VALUES
    (5,'Shipping',1500);
80  INSERT INTO departments(department_id,department_name,location_id) VALUES
    (6,'IT',1400);
81  INSERT INTO departments(department_id,department_name,location_id) VALUES
    (7,'Public Relations',2700);
82  INSERT INTO departments(department_id,department_name,location_id) VALUES
    (8,'Sales',2500);
83  INSERT INTO departments(department_id,department_name,location_id) VALUES
    (9,'Executive',1700);
84  INSERT INTO departments(department_id,department_name,location_id) VALUES
    (10,'Finance',1700);
85  INSERT INTO departments(department_id,department_name,location_id) VALUES
    (11,'Accounting',1700);
86
87  /*Data for the table employees */
88  INSERT INTO employees
    (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
    lary,manager_id,department_id) VALUES
    (100,'Steven','King','steven.king@sqltutorial.org','515.123.4567','1987-
    06-17',4,24000.00,NULL,9);
89  INSERT INTO employees
    (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
    lary,manager_id,department_id) VALUES
    (101,'Neena','Kochhar','neena.kochhar@sqltutorial.org','515.123.4568','1
    989-09-21',5,17000.00,100,9);
90  INSERT INTO employees
    (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
```

```
      lary,manager_id,department_id) VALUES (102,'Lex','De Haan','lex.de
      haan@sqltutorial.org','515.123.4569','1993-01-13',5,17000.00,100,9);
 91  INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (103,'Alexander','Hunold','alexander.hunold@sqltutorial.org','590.423.45
      67','1990-01-03',9,9000.00,102,6);
 92  INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (104,'Bruce','Ernst','bruce.ernst@sqltutorial.org','590.423.4568','1991-
      05-21',9,6000.00,103,6);
 93  INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (105,'David','Austin','david.austin@sqltutorial.org','590.423.4569','199
      7-06-25',9,4800.00,103,6);
 94  INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (106,'Valli','Pataballa','valli.pataballa@sqltutorial.org','590.423.4560
      ','1998-02-05',9,4800.00,103,6);
 95  INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (107,'Diana','Lorentz','diana.lorentz@sqltutorial.org','590.423.5567','1
      999-02-07',9,4200.00,103,6);
 96  INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (108,'Nancy','Greenberg','nancy.greenberg@sqltutorial.org','515.124.4569
      ','1994-08-17',7,12000.00,101,10);
 97  INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (109,'Daniel','Faviet','daniel.faviet@sqltutorial.org','515.124.4169','1
      994-08-16',6,9000.00,108,10);
 98  INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (110,'John','Chen','john.chen@sqltutorial.org','515.124.4269','1997-09-2
      8',6,8200.00,108,10);
 99  INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (111,'Ismael','Sciarra','ismael.sciarra@sqltutorial.org','515.124.4369',
      '1997-09-30',6,7700.00,108,10);
100  INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
```

```
      lary,manager_id,department_id) VALUES (112,'Jose Manuel','Urman','jose
      manuel.urman@sqltutorial.org','515.124.4469','1998-03-07',6,7800.00,108,
      10);
101   INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (113,'Luis','Popp','luis.popp@sqltutorial.org','515.124.4567','1999-12-0
      7',6,6900.00,108,10);
102   INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (114,'Den','Raphaely','den.raphaely@sqltutorial.org','515.127.4561','199
      4-12-07',14,11000.00,100,3);
103   INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (115,'Alexander','Khoo','alexander.khoo@sqltutorial.org','515.127.4562',
      '1995-05-18',13,3100.00,114,3);
104   INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (116,'Shelli','Baida','shelli.baida@sqltutorial.org','515.127.4563','199
      7-12-24',13,2900.00,114,3);
105   INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (117,'Sigal','Tobias','sigal.tobias@sqltutorial.org','515.127.4564','199
      7-07-24',13,2800.00,114,3);
106   INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (118,'Guy','Himuro','guy.himuro@sqltutorial.org','515.127.4565','1998-11
      -15',13,2600.00,114,3);
107   INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (119,'Karen','Colmenares','karen.colmenares@sqltutorial.org','515.127.45
      66','1999-08-10',13,2500.00,114,3);
108   INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (120,'Matthew','Weiss','matthew.weiss@sqltutorial.org','650.123.1234','1
      996-07-18',19,8000.00,100,5);
109   INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (121,'Adam','Fripp','adam.fripp@sqltutorial.org','650.123.2234','1997-04
      -10',19,8200.00,100,5);
110   INSERT INTO employees
```

```
    (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
    lary,manager_id,department_id) VALUES
    (122,'Payam','Kaufling','payam.kaufling@sqltutorial.org','650.123.3234',
    '1995-05-01',19,7900.00,100,5);
111 INSERT INTO employees
    (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
    lary,manager_id,department_id) VALUES
    (123,'Shanta','Vollman','shanta.vollman@sqltutorial.org','650.123.4234',
    '1997-10-10',19,6500.00,100,5);
112 INSERT INTO employees
    (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
    lary,manager_id,department_id) VALUES
    (126,'Irene','Mikkilineni','irene.mikkilineni@sqltutorial.org','650.124.
    1224','1998-09-28',18,2700.00,120,5);
113 INSERT INTO employees
    (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
    lary,manager_id,department_id) VALUES
    (145,'John','Russell','john.russell@sqltutorial.org',NULL,'1996-10-01',1
    5,14000.00,100,8);
114 INSERT INTO employees
    (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
    lary,manager_id,department_id) VALUES
    (146,'Karen','Partners','karen.partners@sqltutorial.org',NULL,'1997-01-0
    5',15,13500.00,100,8);
115 INSERT INTO employees
    (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
    lary,manager_id,department_id) VALUES
    (176,'Jonathon','Taylor','jonathon.taylor@sqltutorial.org',NULL,'1998-03
    -24',16,8600.00,100,8);
116 INSERT INTO employees
    (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
    lary,manager_id,department_id) VALUES
    (177,'Jack','Livingston','jack.livingston@sqltutorial.org',NULL,'1998-04
    -23',16,8400.00,100,8);
117 INSERT INTO employees
    (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
    lary,manager_id,department_id) VALUES
    (178,'Kimberely','Grant','kimberely.grant@sqltutorial.org',NULL,'1999-05
    -24',16,7000.00,100,8);
118 INSERT INTO employees
    (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
    lary,manager_id,department_id) VALUES
    (179,'Charles','Johnson','charles.johnson@sqltutorial.org',NULL,'2000-01
    -04',16,6200.00,100,8);
119 INSERT INTO employees
    (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
    lary,manager_id,department_id) VALUES
    (192,'Sarah','Bell','sarah.bell@sqltutorial.org','650.501.1876','1996-02
    -04',17,4000.00,123,5);
```

```
120 INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (193,'Britney','Everett','britney.everett@sqltutorial.org','650.501.2876
      ','1997-03-03',17,3900.00,123,5);
121 INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (200,'Jennifer','Whalen','jennifer.whalen@sqltutorial.org','515.123.4444
      ','1987-09-17',3,4400.00,101,1);
122 INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (201,'Michael','Hartstein','michael.hartstein@sqltutorial.org','515.123.
      5555','1996-02-17',10,13000.00,100,2);
123 INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (202,'Pat','Fay','pat.fay@sqltutorial.org','603.123.6666','1997-08-17',1
      1,6000.00,201,2);
124 INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (203,'Susan','Mavris','susan.mavris@sqltutorial.org','515.123.7777','199
      4-06-07',8,6500.00,101,4);
125 INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (204,'Hermann','Baer','hermann.baer@sqltutorial.org','515.123.8888','199
      4-06-07',12,10000.00,101,7);
126 INSERT INTO employees
      (employee_id,first_name,last_name,email,phone_number,hire_date,job_id,sa
      lary,manager_id,department_id) VALUES
      (205,'Shelley','Higgins','shelley.higgins@sqltutorial.org','515.123.8080
      ','1994-06-07',2,12000.00,101,11);
127
128 /*Data for the table dependents */
129 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (1,'Penelope','Gietz','Child',206);
130 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (2,'Nick','Higgins','Child',205);
131 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (3,'Ed','Whalen','Child',200);
132 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (4,'Jennifer','King','Child',100);
```

```
133 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (5,'Johnny','Kochhar','Child',101);
134 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (6,'Bette','De Haan','Child',102);
135 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (7,'Grace','Faviet','Child',109);
136 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (8,'Matthew','Chen','Child',110);
137 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (9,'Joe','Sciarra','Child',111);
138 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (10,'Christian','Urman','Child',112);
139 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (11,'Zero','Popp','Child',113);
140 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (12,'Karl','Greenberg','Child',108);
141 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (13,'Uma','Mavris','Child',203);
142 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (14,'Vivien','Hunold','Child',103);
143 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (15,'Cuba','Ernst','Child',104);
144 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (16,'Fred','Austin','Child',105);
145 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (17,'Helen','Pataballa','Child',106);
146 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (18,'Dan','Lorentz','Child',107);
147 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (19,'Bob','Hartstein','Child',201);
148 INSERT INTO dependents
      (dependent_id,first_name,last_name,relationship,employee_id) VALUES
      (20,'Lucille','Fay','Child',202);
149 INSERT INTO dependents
```

```
    (dependent_id,first_name,last_name,relationship,employee_id) VALUES
    (21,'Kirsten','Baer','Child',204);
INSERT INTO dependents
    (dependent_id,first_name,last_name,relationship,employee_id) VALUES
    (22,'Elvis','Khoo','Child',115);
INSERT INTO dependents
    (dependent_id,first_name,last_name,relationship,employee_id) VALUES
    (23,'Sandra','Baida','Child',116);
INSERT INTO dependents
    (dependent_id,first_name,last_name,relationship,employee_id) VALUES
    (24,'Cameron','Tobias','Child',117);
INSERT INTO dependents
    (dependent_id,first_name,last_name,relationship,employee_id) VALUES
    (25,'Kevin','Himuro','Child',118);
INSERT INTO dependents
    (dependent_id,first_name,last_name,relationship,employee_id) VALUES
    (26,'Rip','Colmenares','Child',119);
INSERT INTO dependents
    (dependent_id,first_name,last_name,relationship,employee_id) VALUES
    (27,'Julia','Raphaely','Child',114);
INSERT INTO dependents
    (dependent_id,first_name,last_name,relationship,employee_id) VALUES
    (28,'Woody','Russell','Child',145);
INSERT INTO dependents
    (dependent_id,first_name,last_name,relationship,employee_id) VALUES
    (29,'Alec','Partners','Child',146);
INSERT INTO dependents
    (dependent_id,first_name,last_name,relationship,employee_id) VALUES
    (30,'Sandra','Taylor','Child',176);

-- ========================================
-- TASK 1: 1) SELECT STATEMENTS
-- ========================================
-- A. To get data from all the rows and columns in the employees table:
     SELECT * FROM employees;

-- B. select data from the employee id, first name, last name, and hire
  date of all rows in the employees table:
SELECT employee_id, first_name, last_name, hire_date FROM employees;

-- C. to get the first name, last name, salary, and new salary:
-- D. Increase the salary two times and named as New_SALARY from
  employees table
SELECT first_name, last_name, salary, salary * 2 AS New_SALARY FROM
  employees;

-- ========================================
-- TASK 1: 2) ORDER BY STATEMENTS
-- ========================================
```

```sql
-- A. returns the data from the employee id, first name, last name, hire date, and salary column of the employees table:
SELECT employee_id, first_name, last_name, hire_date, salary FROM employees;

-- B. to sort employees by first names in alphabetical order:
SELECT * FROM employees ORDER BY first_name ASC;

-- C. to sort the employees by the first name in ascending order and the last name in descending order:
SELECT * FROM employees ORDER BY first_name ASC, last_name DESC;

-- D. to sort employees by salary from high to low:
SELECT * FROM employees ORDER BY salary DESC;

-- E. to sort the employees by values in the hire_date column from:
SELECT * FROM employees ORDER BY hire_date ASC;

-- F. sort the employees by the hire dates in descending order:
SELECT * FROM employees ORDER BY hire_date DESC;

-- ========================================
-- TASK 1: 3) DISTINCT STATEMENTS
-- ========================================
-- A. selects the salary data from the salary column of the employees table and sorts them from high to low:
SELECT DISTINCT salary FROM employees ORDER BY salary DESC;

-- B. select unique values from the salary column of the employees table:
SELECT DISTINCT salary FROM employees;

-- C. selects the job id and salary from the employees table:
SELECT job_id, salary FROM employees;

-- D. to remove the duplicate values in job id and salary:
SELECT DISTINCT job_id, salary FROM employees;

-- E. returns the distinct phone numbers of employees:
SELECT DISTINCT phone_number FROM employees;

-- ========================================
-- TASK 1: 4) TOP N STATEMENTS
-- ========================================
-- A. returns all rows in the employees table sorted by the first_name column.
SELECT * FROM employees ORDER BY first_name;

-- B. to return the first 5 rows in the result set returned by the SELECT clause:
```

```
219 SELECT TOP 5 * FROM employees;
220
221 -- C. to return five rows starting from the 4th row:
222 SELECT * FROM employees ORDER BY employee_id OFFSET 3 ROWS FETCH NEXT 5
      ROWS ONLY;
223
224 -- D. gets the top five employees with the highest salaries.
225 SELECT TOP 5 * FROM employees ORDER BY salary DESC;
226
227 -- E. to get employees who have the 2 nd highest salary in the company
228 SELECT * FROM employees ORDER BY salary DESC OFFSET 1 ROWS FETCH NEXT 1
      ROWS ONLY;
229
230 -- ========================================
231 -- TASK 1: 5) WHERE CLAUSE and COMPARISON OPERATORS
232 -- ========================================
233 -- A. query finds employees who have salaries greater than 14,000 and
      sorts the results sets based on the salary in descending order.
234 SELECT * FROM employees WHERE salary > 14000 ORDER BY salary DESC;
235
236 -- B. query finds all employees who work in the department id 5.
237 SELECT * FROM employees WHERE department_id = 5;
238
239 -- C. query finds the employee whose last name is Chen
240 SELECT * FROM employees WHERE last_name = 'Chen';
241
242 -- D. To get all employees who joined the company after January 1st, 1999
243 SELECT * FROM employees WHERE hire_date > '1999-01-01';
244
245 -- E. to find the employees who joined the company in 1999,
246 SELECT * FROM employees WHERE YEAR(hire_date) = 1999;
247
248 -- F. statement finds the employee whose last name is Himuro
249 SELECT * FROM employees WHERE last_name = 'Himuro';
250
251 -- G. the query searches for the string Himuro in the last_name column of
      the employees table.
252 SELECT * FROM employees WHERE last_name LIKE '%Himuro%';
253
254 -- H. to find all employees who do not have phone numbers:
255 SELECT * FROM employees WHERE phone_number IS NULL;
256
257 -- I. returns all employees whose department id is not 8.
258 SELECT * FROM employees WHERE department_id <> 8;
259
260 -- J. finds all employees whose department id is not eight and ten.
261 SELECT * FROM employees WHERE department_id NOT IN (8, 10);
262
263 -- K. to find the employees whose salary is greater than 10,000,
```

```
264 SELECT * FROM employees WHERE salary > 10000;
265
266 -- L. finds employees in department 8 and have the salary greater than
      10,000:
267 SELECT * FROM employees WHERE department_id = 8 AND salary > 10000;
268
269 -- M. the statement below returns all employees whose salaries are less
      than 10,000:
270 SELECT * FROM employees WHERE salary < 10000;
271
272 -- N. finds employees whose salaries are greater than or equal 9,000:
273 SELECT * FROM employees WHERE salary >= 9000;
274
275 -- O. finds employees whose salaries are less than or equal to 9,000:
276 SELECT * FROM employees WHERE salary <= 9000;
277
278 -- ========================================
279 -- TASK 1: 6) WRITE A QUERY FOR:- (ALTER TABLE)
280 -- ========================================
281 -- A. adds a new column named credit_hours to the courses table.
282 -- ALTER TABLE courses ADD credit_hours INT;
283
284 -- B. adds the fee and max_limit columns to the courses table and places
      these columns after the course_name column.
285 -- ALTER TABLE courses ADD fee DECIMAL(10,2), max_limit INT;
286
287 -- C. changes the attribute of the fee column to NOT NULL.
288 -- ALTER TABLE courses ALTER COLUMN fee DECIMAL(10,2) NOT NULL;
289
290 -- D. to remove the fee column of the courses table
291 -- ALTER TABLE courses DROP COLUMN fee;
292
293 -- E. removes the max_limit and credit_hours of the courses table.
294 -- ALTER TABLE courses DROP COLUMN max_limit, credit_hours;
295
296 -- ========================================
297 -- TASK 1: 6) WRITE A QUERY FOR:- SQL foreign key constraint
298 -- ========================================
299 -- A. to add an SQL FOREIGN KEY constraint to the project_milestones
      table to enforce the relationship between the projects and
      project_milestones tables.
300 -- B. Suppose the project_milestones already exists without any
      predefined foreign key and you want to define a FOREIGN KEY constraint
      for the project_id column so write a Query to add a FOREIGN KEY
      constraint to existing table
301 -- ALTER TABLE project_milestones ADD CONSTRAINT fk_project FOREIGN KEY
      (project_id) REFERENCES projects(project_id);
302
303 -- ========================================
```

```sql
-- TASK 2: 1) Logical Operators and Special Operators
-- ========================================
-- Part 1:-
-- A. finds all employees whose salaries are greater than 5,000 and less than 7,000:
SELECT * FROM employees WHERE salary > 5000 AND salary < 7000;

-- B. finds employees whose salary is either 7,000 or 8,000:
SELECT * FROM employees WHERE salary IN (7000, 8000);

-- C. finds all employees who do not have a phone number:
SELECT * FROM employees WHERE phone_number IS NULL;

-- D. finds all employees whose salaries are between 9,000 and 12,000.
SELECT * FROM employees WHERE salary BETWEEN 9000 AND 12000;

-- E. finds all employees who work in the department id 8 or 9.
SELECT * FROM employees WHERE department_id IN (8, 9);

-- F. finds all employees whose first name starts with the string jo
SELECT * FROM employees WHERE first_name LIKE 'jo%';

-- G. finds all employees with the first names whose the second character is h
SELECT * FROM employees WHERE first_name LIKE '_h%';

-- H. finds all employees whose salaries are greater than all salaries of employees in the department 8:
SELECT * FROM employees WHERE salary > ALL (SELECT salary FROM employees WHERE department_id = 8);

-- Part 2:-
-- A. finds all employees whose salaries are greater than the average salary of every department:
SELECT * FROM employees WHERE salary > ALL (SELECT AVG(salary) FROM employees GROUP BY department_id);

-- B. finds all employees who have dependents:
SELECT * FROM employees e WHERE EXISTS (SELECT 1 FROM dependents d WHERE d.employee_id = e.employee_id);

-- C. to find all employees whose salaries are between 2,500 and 2,900:
SELECT * FROM employees WHERE salary BETWEEN 2500 AND 2900;

-- D. to find all employees whose salaries are not in the range of 2,500 and 2,900:
SELECT * FROM employees WHERE salary NOT BETWEEN 2500 AND 2900;

-- E. to find all employees who joined the company between January 1,
```

```
      1999 , and December 31, 2000 :
345 SELECT * FROM employees WHERE hire_date BETWEEN '1999-01-01' AND
      '2000-12-31';
346
347 -- F. to find employees who have not joined the company from January 1,
      1989 to December 31, 1999 :
348 SELECT * FROM employees WHERE hire_date NOT BETWEEN '1989-01-01' AND
      '1999-12-31';
349
350 -- G. to find employees who joined the company between 1990 and 1993:
351 SELECT * FROM employees WHERE YEAR(hire_date) BETWEEN 1990 AND 1993;
352
353 -- Part 3:-
354 -- A. to find all employees whose first names start with Da
355 SELECT * FROM employees WHERE first_name LIKE 'Da%';
356
357 -- B. to find all employees whose first names end with er
358 SELECT * FROM employees WHERE first_name LIKE '%er';
359
360 -- C. to find employees whose last names contain the word an :
361 SELECT * FROM employees WHERE last_name LIKE '%an%';
362
363 -- D. retrieves employees whose first names start with Jo and are
      followed by at most 2 characters:
364 SELECT * FROM employees WHERE first_name LIKE 'Jo_' OR first_name LIKE
      'Jo__';
365
366 -- E. to find employees whose first names start with any number of
      characters and are followed by at most one character:
367 SELECT * FROM employees WHERE first_name LIKE '%_';
368
369 -- F. to find all employees whose first names start with the letter S but
      not start with Sh :
370 SELECT * FROM employees WHERE first_name LIKE 'S%' AND first_name NOT
      LIKE 'Sh%';
371
372 -- Part 4:-
373 -- A. retrieves all employees who work in the department id 5.
374 SELECT * FROM employees WHERE department_id = 5;
375
376 -- B. To get the employees who work in the department id 5 and with a
      salary not greater than 5000.
377 SELECT * FROM employees WHERE department_id = 5 AND salary <= 5000;
378
379 -- C. statement gets all the employees who are not working in the
      departments 1, 2, or 3.
380 SELECT * FROM employees WHERE department_id NOT IN (1, 2, 3);
381
382 -- D. retrieves all the employees whose first names do not start with the
```

```
    letter D.
383 SELECT * FROM employees WHERE first_name NOT LIKE 'D%';
384
385 -- E. to get employees whose salaries are not between 5,000 and 1,000.
386 SELECT * FROM employees WHERE salary NOT BETWEEN 1000 AND 5000;
387
388 -- Part 5:-
389 -- A. Write a query to get the employees who do not have any dependents
    by above image
390 SELECT * FROM employees e WHERE NOT EXISTS (SELECT 1 FROM dependents d
    WHERE d.employee_id = e.employee_id);
391
392 -- B. To find all employees who do not have the phone numbers
393 SELECT * FROM employees WHERE phone_number IS NULL;
394
395 -- C. To find all employees who have phone numbers
396 SELECT * FROM employees WHERE phone_number IS NOT NULL;
397
398 -- ========================================
399 -- TASK 3: JOINS:-
400 -- ========================================
401 -- 1) Write a Query to
402 -- A. To get the information of the department id 1,2, and 3
403 SELECT * FROM departments WHERE department_id IN (1, 2, 3);
404
405 -- B. To get the information of employees who work in the department id
    1, 2 and 3
406 SELECT * FROM employees WHERE department_id IN (1, 2, 3);
407
408 -- C. Write a Query to get the first name, last name, job title, and
    department name of employees who work in department id 1, 2, and 3.
409 SELECT e.first_name, e.last_name, j.job_title, d.department_name
410 FROM employees e
411 INNER JOIN departments d ON e.department_id = d.department_id
412 INNER JOIN jobs j ON e.job_id = j.job_id
413 WHERE d.department_id IN (1, 2, 3);
414
415 -- SQL LEFT JOIN clause
416 -- A. To query the country names of US, UK, and China
417 SELECT country_name FROM countries WHERE country_id IN ('US', 'UK', 'CN');
418
419 -- B. query retrieves the locations located in the US, UK and China:
420 SELECT * FROM locations WHERE country_id IN ('US', 'UK', 'CN');
421
422 -- C. To join the countries table with the locations table
423 SELECT * FROM countries c LEFT JOIN locations l ON c.country_id =
    l.country_id;
424
425 -- D. to find the country that does not have any locations in the
```

```
    locations table
426 SELECT c.country_name FROM countries c LEFT JOIN locations l ON
    c.country_id = l.country_id WHERE l.location_id IS NULL;
427
428 -- Write a query to join 3 tables: regions, countries, and locations
429 SELECT r.region_name, c.country_name, l.city
430 FROM regions r
431 LEFT JOIN countries c ON r.region_id = c.region_id
432 LEFT JOIN locations l ON c.country_id = l.country_id;
433
434 -- SQL self-join
435 -- The manager_id column specifies the manager of an employee. Write a
    query statement to joins the employees table to itself to query the
    information of who reports to whom.
436 SELECT e.first_name + ' ' + e.last_name AS employee, m.first_name + ' ' +
    m.last_name AS manager
437 FROM employees e
438 INNER JOIN employees m ON e.manager_id = m.employee_id;
439
440 -- Now write a Query To include the president in the result set:-
441 SELECT e.first_name + ' ' + e.last_name AS employee, m.first_name + ' ' +
    m.last_name AS manager
442 FROM employees e
443 LEFT JOIN employees m ON e.manager_id = m.employee_id;
444
445 -- SQL FULL OUTER JOIN clause
446 -- A. Write a query to returns each fruit that is in a basket and each
    basket that has a fruit, but also returns each fruit that is not in any
    basket and each basket that does not have any fruit.
447 -- SELECT * FROM fruits f FULL OUTER JOIN baskets b ON f.basket_id =
    b.basket_id;
448
449 -- B. Write a query to find the empty basket, which does not store any
    fruit
450 -- SELECT * FROM baskets b LEFT JOIN fruits f ON b.basket_id = f.basket_
    id WHERE f.fruit_id IS NULL;
451
452 -- C. Write a query which fruit is not in any basket
453 -- SELECT * FROM fruits f LEFT JOIN baskets b ON f.basket_id = b.basket_
    id WHERE b.basket_id IS NULL;
454
455 -- SQL CROSS JOIN clause
456 -- Write a Query To find the all possible sales channels that a sales
    organization
457 -- SELECT * FROM sales_organization CROSS JOIN sales_channel;
458
459 -- ========================================
460 -- TASK 4: SQL GROUP BY clause
461 -- ========================================
```

```
462 -- A. to group the values in department_id column of the employees table:
463 SELECT department_id FROM employees GROUP BY department_id;
464
465 -- B. to count the number of employees by department:
466 SELECT department_id, COUNT(*) AS num_employees FROM employees GROUP BY
      department_id;
467
468 -- C. returns the number of employees by department
469 SELECT department_id, COUNT(employee_id) AS num_employees FROM employees
      GROUP BY department_id;
470
471 -- D. to sort the departments by headcount:
472 SELECT department_id, COUNT(*) AS num_employees FROM employees GROUP BY
      department_id ORDER BY num_employees DESC;
473
474 -- E. to find departments with headcounts are greater than 5:
475 SELECT department_id, COUNT(*) AS num_employees FROM employees GROUP BY
      department_id HAVING COUNT(*) > 5;
476
477 -- F. returns the minimum, maximum and average salary of employees in
      each department.
478 SELECT department_id, MIN(salary) AS min_salary, MAX(salary) AS
      max_salary, AVG(salary) AS avg_salary FROM employees GROUP BY
      department_id;
479
480 -- G. To get the total salary per department,
481 SELECT department_id, SUM(salary) AS total_salary FROM employees GROUP BY
      department_id;
482
483 -- H. groups rows with the same values both department_id and job_id
      columns in the same group then return the rows for each of these groups
484 SELECT department_id, job_id, COUNT(*) FROM employees GROUP BY
      department_id, job_id;
485
486 -- SQL HAVING clause
487 -- A. To get the managers and their direct reports, and to group
      employees by the managers and to count the direct reports.
488 SELECT manager_id, COUNT(employee_id) AS direct_reports FROM employees
      WHERE manager_id IS NOT NULL GROUP BY manager_id;
489
490 -- B. To find the managers who have at least five direct reports
491 SELECT manager_id, COUNT(employee_id) AS direct_reports FROM employees
      WHERE manager_id IS NOT NULL GROUP BY manager_id HAVING COUNT
      (employee_id) >= 5;
492
493 -- C. calculates the sum of salary that the company pays for each
      department and selects only the departments with the sum of salary
      between 20000 and 30000.
494 SELECT department_id, SUM(salary) AS total_salary FROM employees GROUP BY
```

```
      department_id HAVING SUM(salary) BETWEEN 20000 AND 30000;
495
496  -- D. To find the department that has employees with the lowest salary
       greater than 10000
497  SELECT department_id FROM employees GROUP BY department_id HAVING MIN
       (salary) > 10000;
498
499  -- E. To find the departments that have the average salaries of employees
       between 5000 and 7000
500  SELECT department_id FROM employees GROUP BY department_id HAVING AVG
       (salary) BETWEEN 5000 AND 7000;
501
502  -- ========================================
503  -- TASK 5 (Other Queries)
504  -- ========================================
505  -- 1) SQL UNION operator
506  -- Write a Query to combine the first name and last name of employees and
       dependents
507  SELECT first_name, last_name FROM employees UNION SELECT first_name,
       last_name FROM dependents;
508
509  -- 2) SQL INTERSECT operator
510  -- Write a Query to Applies the INTERSECT operator to the A and B tables
       and sorts the combined result set by the id column in descending order.
511  -- SELECT id FROM A INTERSECT SELECT id FROM B ORDER BY id DESC;
512
513  -- 3) SQL EXISTS operator
514  -- A. finds all employees who have at least one dependent.
515  SELECT * FROM employees e WHERE EXISTS (SELECT 1 FROM dependents d WHERE
       d.employee_id = e.employee_id);
516
517  -- B . finds employees who do not have any dependents:
518  SELECT * FROM employees e WHERE NOT EXISTS (SELECT 1 FROM dependents d
       WHERE d.employee_id = e.employee_id);
519
520  -- 4) SQL CASE expression
521  -- A. Suppose the current year is 2000. How to use the simple CASE
       expression to get the work anniversaries of employees by
522  SELECT first_name, last_name, CASE WHEN YEAR(hire_date) = 2000 THEN
       'Anniversary this year' ELSE 'No anniversary this year' END AS
       anniversary_status FROM employees;
523
524  -- B. Write a Query If the salary is less than 3000, the CASE expression
       returns "Low". If the salary is between 3000 and 5000, it returns
       "average". When the salary is greater than 5000, the CASE expression
       returns "High".
525  SELECT first_name, last_name, salary,
526  CASE
527      WHEN salary < 3000 THEN 'Low'
```

```sql
    WHEN salary BETWEEN 3000 AND 5000 THEN 'average'
    WHEN salary > 5000 THEN 'High'
END AS salary_level FROM employees;

-- 5) SQL UPDATE statement
-- Write a Query to update Sarah's last name from Bell to Lopez
UPDATE employees SET last_name = 'Lopez' WHERE first_name = 'Sarah' AND last_name = 'Bell';

-- How to make sure that the last names of children are always matched with the last name of parents in the employees table,
UPDATE dependents SET last_name = (SELECT last_name FROM employees WHERE employees.employee_id = dependents.employee_id);

-- ========================================
-- FINAL TASK (Advanced Queries) SQL SUBQUERY
-- ========================================
-- A. Combine Above two queries using subquery inorder find all departments located at the location whose id is 1700 and find all employees that belong to the location 1700 by using the department id list of the previous query
SELECT employee_id, first_name, last_name FROM employees WHERE department_id IN (SELECT department_id FROM departments WHERE location_id = 1700) ORDER BY first_name, last_name;

-- B. to find all employees who do not locate at the location 1700:
SELECT employee_id, first_name, last_name FROM employees WHERE department_id NOT IN (SELECT department_id FROM departments WHERE location_id = 1700) ORDER BY first_name, last_name;

-- C. finds the employees who have the highest salary:
SELECT * FROM employees WHERE salary = (SELECT MAX(salary) FROM employees);

-- D. finds all employees who salaries are greater than the average salary of all employees:
SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);

-- E. finds all departments which have at least one employee with the salary is greater than 10,000:
SELECT * FROM departments WHERE department_id IN (SELECT DISTINCT department_id FROM employees WHERE salary > 10000);

-- F. finds all departments that do not have any employee with the salary greater than 10,000:
SELECT * FROM departments WHERE department_id NOT IN (SELECT DISTINCT department_id FROM employees WHERE salary > 10000 AND department_id IS NOT NULL);
```

```
559
560 -- G. to find the lowest salary by department:
561 SELECT department_id, MIN(salary) AS lowest_salary FROM employees GROUP
      BY department_id;
562
563 -- H. finds all employees whose salaries are greater than the lowest
      salary of every department:
564 SELECT * FROM employees WHERE salary > ALL (SELECT MIN(salary) FROM
      employees GROUP BY department_id);
565
566 -- I. finds all employees whose salaries are greater than or equal to the
      highest salary of every department
567 SELECT * FROM employees WHERE salary >= ALL (SELECT MAX(salary) FROM
      employees GROUP BY department_id);
568
569 -- J. returns the average salary of every department
570 SELECT department_id, AVG(salary) AS avg_salary FROM employees GROUP BY
      department_id;
571
572 -- K. to calculate the average of average salary of departments :
573 SELECT AVG(avg_salary) FROM (SELECT AVG(salary) AS avg_salary FROM
      employees GROUP BY department_id) t;
574
575 -- L. finds the salaries of all employees, their average salary, and the
      difference between the salary of each employee and the average salary.
576 SELECT first_name, last_name, salary, (SELECT AVG(salary) FROM employees)
      AS average_salary, salary - (SELECT AVG(salary) FROM employees) AS
      salary_difference FROM employees;
577
578 /************************************* TASKS
      ********************************************************************/
579
580 /* SAME TASK AS FROM THE 159-575 */
581 -- ========================================
582 -- TASK 1: 1) SELECT STATEMENTS
583 -- ========================================
584 -- A. To get data from all the rows and columns in the employees table:
585 SELECT * FROM employees;
586
587 -- B. select data from the employee id, first name, last name, and hire
      date of all rows in the employees table:
588 SELECT employee_id, first_name, last_name, hire_date FROM employees;
589
590 -- C. to get the first name, last name, salary, and new salary:
591 -- D. Increase the salary two times and named as New_SALARY from
      employees table
592 SELECT first_name, last_name, salary, salary * 2 AS New_SALARY FROM
      employees;
593
```

```sql
-- ========================================
-- TASK 1: 2) ORDER BY STATEMENTS
-- ========================================
-- A. returns the data from the employee id, first name, last name, hire date, and salary column of the employees table:
SELECT employee_id, first_name, last_name, hire_date, salary FROM employees;

-- B. to sort employees by first names in alphabetical order:
SELECT * FROM employees ORDER BY first_name ASC;

-- C. to sort the employees by the first name in ascending order and the last name in descending order:
SELECT * FROM employees ORDER BY first_name ASC, last_name DESC;

-- D. to sort employees by salary from high to low:
SELECT * FROM employees ORDER BY salary DESC;

-- E. to sort the employees by values in the hire_date column from:
SELECT * FROM employees ORDER BY hire_date ASC;

-- F. sort the employees by the hire dates in descending order:
SELECT * FROM employees ORDER BY hire_date DESC;

-- ========================================
-- TASK 1: 3) DISTINCT STATEMENTS
-- ========================================
-- A. selects the salary data from the salary column of the employees table and sorts them from high to low:
SELECT DISTINCT salary FROM employees ORDER BY salary DESC;

-- B. select unique values from the salary column of the employees table:
SELECT DISTINCT salary FROM employees;

-- C. selects the job id and salary from the employees table:
SELECT job_id, salary FROM employees;

-- D. to remove the duplicate values in job id and salary:
SELECT DISTINCT job_id, salary FROM employees;

-- E. returns the distinct phone numbers of employees:
SELECT DISTINCT phone_number FROM employees;

-- ========================================
-- TASK 1: 4) TOP N STATEMENTS
-- ========================================
-- A. returns all rows in the employees table sorted by the first_name column.
SELECT * FROM employees ORDER BY first_name;
```

```sql

-- B. to return the first 5 rows in the result set returned by the SELECT clause:
SELECT TOP 5 * FROM employees;

-- C. to return five rows starting from the 4th row:
SELECT * FROM employees ORDER BY employee_id OFFSET 3 ROWS FETCH NEXT 5 ROWS ONLY;

-- D. gets the top five employees with the highest salaries.
SELECT TOP 5 * FROM employees ORDER BY salary DESC;

-- E. to get employees who have the 2 nd highest salary in the company
SELECT * FROM employees ORDER BY salary DESC OFFSET 1 ROWS FETCH NEXT 1 ROWS ONLY;

-- ========================================
-- TASK 1: 5) WHERE CLAUSE and COMPARISON OPERATORS
-- ========================================
-- A. query finds employees who have salaries greater than 14,000 and sorts the results sets based on the salary in descending order.
SELECT * FROM employees WHERE salary > 14000 ORDER BY salary DESC;

-- B. query finds all employees who work in the department id 5.
SELECT * FROM employees WHERE department_id = 5;

-- C. query finds the employee whose last name is Chen
SELECT * FROM employees WHERE last_name = 'Chen';

-- D. To get all employees who joined the company after January 1st, 1999
SELECT * FROM employees WHERE hire_date > '1999-01-01';

-- E. to find the employees who joined the company in 1999,
SELECT * FROM employees WHERE YEAR(hire_date) = 1999;

-- F. statement finds the employee whose last name is Himuro
SELECT * FROM employees WHERE last_name = 'Himuro';

-- G. the query searches for the string Himuro in the last_name column of the employees table.
SELECT * FROM employees WHERE last_name LIKE '%Himuro%';

-- H. to find all employees who do not have phone numbers:
SELECT * FROM employees WHERE phone_number IS NULL;

-- I. returns all employees whose department id is not 8.
SELECT * FROM employees WHERE department_id <> 8;

-- J. finds all employees whose department id is not eight and ten.
```

```sql
SELECT * FROM employees WHERE department_id NOT IN (8, 10);

-- K. to find the employees whose salary is greater than 10,000,
SELECT * FROM employees WHERE salary > 10000;

-- L. finds employees in department 8 and have the salary greater than 10,000:
SELECT * FROM employees WHERE department_id = 8 AND salary > 10000;

-- M. the statement below returns all employees whose salaries are less than 10,000:
SELECT * FROM employees WHERE salary < 10000;

-- N. finds employees whose salaries are greater than or equal 9,000:
SELECT * FROM employees WHERE salary >= 9000;

-- O. finds employees whose salaries are less than or equal to 9,000:
SELECT * FROM employees WHERE salary <= 9000;

-- ========================================
-- TASK 1: 6) WRITE A QUERY FOR:- (ALTER TABLE)
-- ========================================
-- A. adds a new column named credit_hours to the courses table.
-- ALTER TABLE courses ADD credit_hours INT;

-- B. adds the fee and max_limit columns to the courses table and places these columns after the course_name column.
-- ALTER TABLE courses ADD fee DECIMAL(10,2), max_limit INT;

-- C. changes the attribute of the fee column to NOT NULL.
-- ALTER TABLE courses ALTER COLUMN fee DECIMAL(10,2) NOT NULL;

-- D. to remove the fee column of the courses table
-- ALTER TABLE courses DROP COLUMN fee;

-- E. removes the max_limit and credit_hours of the courses table.
-- ALTER TABLE courses DROP COLUMN max_limit, credit_hours;

-- ========================================
-- TASK 1: 6) WRITE A QUERY FOR:- SQL foreign key constraint
-- ========================================
-- A. to add an SQL FOREIGN KEY constraint to the project_milestones table to enforce the relationship between the projects and project_milestones tables.
-- B. Suppose the project_milestones already exists without any predefined foreign key and you want to define a FOREIGN KEY constraint for the project_id column so write a Query to add a FOREIGN KEY constraint to existing table
-- ALTER TABLE project_milestones ADD CONSTRAINT fk_project FOREIGN KEY
```

```
    (project_id) REFERENCES projects(project_id);

-- ========================================
-- TASK 2: 1) Logical Operators and Special Operators
-- ========================================
-- Part 1:-
-- A. finds all employees whose salaries are greater than 5,000 and less than 7,000:
SELECT * FROM employees WHERE salary > 5000 AND salary < 7000;

-- B. finds employees whose salary is either 7,000 or 8,000:
SELECT * FROM employees WHERE salary IN (7000, 8000);

-- C. finds all employees who do not have a phone number:
SELECT * FROM employees WHERE phone_number IS NULL;

-- D. finds all employees whose salaries are between 9,000 and 12,000.
SELECT * FROM employees WHERE salary BETWEEN 9000 AND 12000;

-- E. finds all employees who work in the department id 8 or 9.
SELECT * FROM employees WHERE department_id IN (8, 9);

-- F. finds all employees whose first name starts with the string jo
SELECT * FROM employees WHERE first_name LIKE 'jo%';

-- G. finds all employees with the first names whose the second character is h
SELECT * FROM employees WHERE first_name LIKE '_h%';

-- H. finds all employees whose salaries are greater than all salaries of employees in the department 8:
SELECT * FROM employees WHERE salary > ALL (SELECT salary FROM employees WHERE department_id = 8);

-- Part 2:-
-- A. finds all employees whose salaries are greater than the average salary of every department:
SELECT * FROM employees WHERE salary > ALL (SELECT AVG(salary) FROM employees GROUP BY department_id);

-- B. finds all employees who have dependents:
SELECT * FROM employees e WHERE EXISTS (SELECT 1 FROM dependents d WHERE d.employee_id = e.employee_id);

-- C. to find all employees whose salaries are between 2,500 and 2,900:
SELECT * FROM employees WHERE salary BETWEEN 2500 AND 2900;

-- D. to find all employees whose salaries are not in the range of 2,500 and 2,900:
```

```sql
SELECT * FROM employees WHERE salary NOT BETWEEN 2500 AND 2900;

-- E. to find all employees who joined the company between January 1, 1999 , and December 31, 2000 :
SELECT * FROM employees WHERE hire_date BETWEEN '1999-01-01' AND '2000-12-31';

-- F. to find employees who have not joined the company from January 1, 1989 to December 31, 1999 :
SELECT * FROM employees WHERE hire_date NOT BETWEEN '1989-01-01' AND '1999-12-31';

-- G. to find employees who joined the company between 1990 and 1993:
SELECT * FROM employees WHERE YEAR(hire_date) BETWEEN 1990 AND 1993;

-- Part 3:-
-- A. to find all employees whose first names start with Da
SELECT * FROM employees WHERE first_name LIKE 'Da%';

-- B. to find all employees whose first names end with er
SELECT * FROM employees WHERE first_name LIKE '%er';

-- C. to find employees whose last names contain the word an :
SELECT * FROM employees WHERE last_name LIKE '%an%';

-- D. retrieves employees whose first names start with Jo and are followed by at most 2 characters:
SELECT * FROM employees WHERE first_name LIKE 'Jo_' OR first_name LIKE 'Jo__';

-- E. to find employees whose first names start with any number of characters and are followed by at most one character:
SELECT * FROM employees WHERE first_name LIKE '%_';

-- F. to find all employees whose first names start with the letter S but not start with Sh :
SELECT * FROM employees WHERE first_name LIKE 'S%' AND first_name NOT LIKE 'Sh%';

-- Part 4:-
-- A. retrieves all employees who work in the department id 5.
SELECT * FROM employees WHERE department_id = 5;

-- B. To get the employees who work in the department id 5 and with a salary not greater than 5000.
SELECT * FROM employees WHERE department_id = 5 AND salary <= 5000;

-- C. statement gets all the employees who are not working in the departments 1, 2, or 3.
```

```sql
SELECT * FROM employees WHERE department_id NOT IN (1, 2, 3);

-- D. retrieves all the employees whose first names do not start with the letter D.
SELECT * FROM employees WHERE first_name NOT LIKE 'D%';

-- E. to get employees whose salaries are not between 5,000 and 1,000.
SELECT * FROM employees WHERE salary NOT BETWEEN 1000 AND 5000;

-- Part 5:-
-- A. Write a query to get the employees who do not have any dependents by above image
SELECT * FROM employees e WHERE NOT EXISTS (SELECT 1 FROM dependents d WHERE d.employee_id = e.employee_id);

-- B. To find all employees who do not have the phone numbers
SELECT * FROM employees WHERE phone_number IS NULL;

-- C. To find all employees who have phone numbers
SELECT * FROM employees WHERE phone_number IS NOT NULL;

-- ========================================
-- TASK 3: JOINS:-
-- ========================================
-- 1) Write a Query to
-- A. To get the information of the department id 1,2, and 3
SELECT * FROM departments WHERE department_id IN (1, 2, 3);

-- B. To get the information of employees who work in the department id 1, 2 and 3
SELECT * FROM employees WHERE department_id IN (1, 2, 3);

-- C. Write a Query to get the first name, last name, job title, and department name of employees who work in department id 1, 2, and 3.
SELECT e.first_name, e.last_name, j.job_title, d.department_name
FROM employees e
INNER JOIN departments d ON e.department_id = d.department_id
INNER JOIN jobs j ON e.job_id = j.job_id
WHERE d.department_id IN (1, 2, 3);

-- SQL LEFT JOIN clause
-- A. To query the country names of US, UK, and China
SELECT country_name FROM countries WHERE country_id IN ('US', 'UK', 'CN');

-- B. query retrieves the locations located in the US, UK and China:
SELECT * FROM locations WHERE country_id IN ('US', 'UK', 'CN');

-- C. To join the countries table with the locations table
SELECT * FROM countries c LEFT JOIN locations l ON c.country_id =
```

```
    l.country_id;
845
846 -- D. to find the country that does not have any locations in the locations table
847 SELECT c.country_name FROM countries c LEFT JOIN locations l ON c.country_id = l.country_id WHERE l.location_id IS NULL;
848
849 -- Write a query to join 3 tables: regions, countries, and locations
850 SELECT r.region_name, c.country_name, l.city
851 FROM regions r
852 LEFT JOIN countries c ON r.region_id = c.region_id
853 LEFT JOIN locations l ON c.country_id = l.country_id;
854
855 -- SQL self-join
856 -- The manager_id column specifies the manager of an employee. Write a query statement to joins the employees table to itself to query the information of who reports to whom.
857 SELECT e.first_name + ' ' + e.last_name AS employee, m.first_name + ' ' + m.last_name AS manager
858 FROM employees e
859 INNER JOIN employees m ON e.manager_id = m.employee_id;
860
861 -- Now write a Query To include the president in the result set:-
862 SELECT e.first_name + ' ' + e.last_name AS employee, m.first_name + ' ' + m.last_name AS manager
863 FROM employees e
864 LEFT JOIN employees m ON e.manager_id = m.employee_id;
865
866 -- SQL FULL OUTER JOIN clause
867 -- A. Write a query to returns each fruit that is in a basket and each basket that has a fruit, but also returns each fruit that is not in any basket and each basket that does not have any fruit.
868 -- SELECT * FROM fruits f FULL OUTER JOIN baskets b ON f.basket_id = b.basket_id;
869
870 -- B. Write a query to find the empty basket, which does not store any fruit
871 -- SELECT * FROM baskets b LEFT JOIN fruits f ON b.basket_id = f.basket_id WHERE f.fruit_id IS NULL;
872
873 -- C. Write a query which fruit is not in any basket
874 -- SELECT * FROM fruits f LEFT JOIN baskets b ON f.basket_id = b.basket_id WHERE b.basket_id IS NULL;
875
876 -- SQL CROSS JOIN clause
877 -- Write a Query To find the all possible sales channels that a sales organization
878 -- SELECT * FROM sales_organization CROSS JOIN sales_channel;
879
```

```sql
-- ========================================
-- TASK 4: SQL GROUP BY clause
-- ========================================
-- A. to group the values in department_id column of the employees table:
SELECT department_id FROM employees GROUP BY department_id;

-- B. to count the number of employees by department:
SELECT department_id, COUNT(*) AS num_employees FROM employees GROUP BY
  department_id;

-- C. returns the number of employees by department
SELECT department_id, COUNT(employee_id) AS num_employees FROM employees
  GROUP BY department_id;

-- D. to sort the departments by headcount:
SELECT department_id, COUNT(*) AS num_employees FROM employees GROUP BY
  department_id ORDER BY num_employees DESC;

-- E. to find departments with headcounts are greater than 5:
SELECT department_id, COUNT(*) AS num_employees FROM employees GROUP BY
  department_id HAVING COUNT(*) > 5;

-- F. returns the minimum, maximum and average salary of employees in
  each department.
SELECT department_id, MIN(salary) AS min_salary, MAX(salary) AS
  max_salary, AVG(salary) AS avg_salary FROM employees GROUP BY
  department_id;

-- G. To get the total salary per department,
SELECT department_id, SUM(salary) AS total_salary FROM employees GROUP BY
  department_id;

-- H. groups rows with the same values both department_id and job_id
  columns in the same group then return the rows for each of these groups
SELECT department_id, job_id, COUNT(*) FROM employees GROUP BY
  department_id, job_id;

-- SQL HAVING clause
-- A. To get the managers and their direct reports, and to group
  employees by the managers and to count the direct reports.
SELECT manager_id, COUNT(employee_id) AS direct_reports FROM employees
  WHERE manager_id IS NOT NULL GROUP BY manager_id;

-- B. To find the managers who have at least five direct reports
SELECT manager_id, COUNT(employee_id) AS direct_reports FROM employees
  WHERE manager_id IS NOT NULL GROUP BY manager_id HAVING COUNT
  (employee_id) >= 5;

-- C. calculates the sum of salary that the company pays for each
```

```
      department and selects only the departments with the sum of salary
      between 20000 and 30000.
915 SELECT department_id, SUM(salary) AS total_salary FROM employees GROUP BY
      department_id HAVING SUM(salary) BETWEEN 20000 AND 30000;
916
917 -- D. To find the department that has employees with the lowest salary
      greater than 10000
918 SELECT department_id FROM employees GROUP BY department_id HAVING MIN
      (salary) > 10000;
919
920 -- E. To find the departments that have the average salaries of employees
      between 5000 and 7000
921 SELECT department_id FROM employees GROUP BY department_id HAVING AVG
      (salary) BETWEEN 5000 AND 7000;
922
923 -- ========================================
924 -- TASK 5 (Other Queries)
925 -- ========================================
926 -- 1) SQL UNION operator
927 -- Write a Query to combine the first name and last name of employees and
      dependents
928 SELECT first_name, last_name FROM employees UNION SELECT first_name,
      last_name FROM dependents;
929
930 -- 2) SQL INTERSECT operator
931 -- Write a Query to Applies the INTERSECT operator to the A and B tables
      and sorts the combined result set by the id column in descending order.
932 -- SELECT id FROM A INTERSECT SELECT id FROM B ORDER BY id DESC;
933
934 -- 3) SQL EXISTS operator
935 -- A. finds all employees who have at least one dependent.
936 SELECT * FROM employees e WHERE EXISTS (SELECT 1 FROM dependents d WHERE
      d.employee_id = e.employee_id);
937
938 -- B . finds employees who do not have any dependents:
939 SELECT * FROM employees e WHERE NOT EXISTS (SELECT 1 FROM dependents d
      WHERE d.employee_id = e.employee_id);
940
941 -- 4) SQL CASE expression
942 -- A. Suppose the current year is 2000. How to use the simple CASE
      expression to get the work anniversaries of employees by
943 SELECT first_name, last_name, CASE WHEN YEAR(hire_date) = 2000 THEN
      'Anniversary this year' ELSE 'No anniversary this year' END AS
      anniversary_status FROM employees;
944
945 -- B. Write a Query If the salary is less than 3000, the CASE expression
      returns "Low". If the salary is between 3000 and 5000, it returns
      "average". When the salary is greater than 5000, the CASE expression
      returns "High".
```

```sql
SELECT first_name, last_name, salary,
CASE
    WHEN salary < 3000 THEN 'Low'
    WHEN salary BETWEEN 3000 AND 5000 THEN 'average'
    WHEN salary > 5000 THEN 'High'
END AS salary_level FROM employees;

-- 5) SQL UPDATE statement
-- Write a Query to update Sarah's last name from Bell to Lopez
UPDATE employees SET last_name = 'Lopez' WHERE first_name = 'Sarah' AND
  last_name = 'Bell';

-- How to make sure that the last names of children are always matched
  with the last name of parents in the employees table,
UPDATE dependents SET last_name = (SELECT last_name FROM employees WHERE
  employees.employee_id = dependents.employee_id);

-- ========================================
-- FINAL TASK (Advanced Queries) SQL SUBQUERY
-- ========================================
-- A. Combine Above two queries using subquery inorder find all
  departments located at the location whose id is 1700 and find all
  employees that belong to the location 1700 by using the department id
  list of the previous query
SELECT employee_id, first_name, last_name FROM employees WHERE
  department_id IN (SELECT department_id FROM departments WHERE
  location_id = 1700) ORDER BY first_name, last_name;

-- B. to find all employees who do not locate at the location 1700:
SELECT employee_id, first_name, last_name FROM employees WHERE
  department_id NOT IN (SELECT department_id FROM departments WHERE
  location_id = 1700) ORDER BY first_name, last_name;

-- C. finds the employees who have the highest salary:
SELECT * FROM employees WHERE salary = (SELECT MAX(salary) FROM
  employees);

-- D. finds all employees who salaries are greater than the average
  salary of all employees:
SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM
  employees);

-- E. finds all departments which have at least one employee with the
  salary is greater than 10,000:
SELECT * FROM departments WHERE department_id IN (SELECT DISTINCT
  department_id FROM employees WHERE salary > 10000);

-- F. finds all departments that do not have any employee with the salary
  greater than 10,000:
```

```sql
SELECT * FROM departments WHERE department_id NOT IN (SELECT DISTINCT department_id FROM employees WHERE salary > 10000 AND department_id IS NOT NULL);

-- G. to find the lowest salary by department:
SELECT department_id, MIN(salary) AS lowest_salary FROM employees GROUP BY department_id;

-- H. finds all employees whose salaries are greater than the lowest salary of every department:
SELECT * FROM employees WHERE salary > ALL (SELECT MIN(salary) FROM employees GROUP BY department_id);

-- I. finds all employees whose salaries are greater than or equal to the highest salary of every department
SELECT * FROM employees WHERE salary >= ALL (SELECT MAX(salary) FROM employees GROUP BY department_id);

-- J. returns the average salary of every department
SELECT department_id, AVG(salary) AS avg_salary FROM employees GROUP BY department_id;

-- K. to calculate the average of average salary of departments :
SELECT AVG(avg_salary) FROM (SELECT AVG(salary) AS avg_salary FROM employees GROUP BY department_id) t;

-- L. finds the salaries of all employees, their average salary, and the difference between the salary of each employee and the average salary.
SELECT first_name, last_name, salary, (SELECT AVG(salary) FROM employees) AS average_salary, salary - (SELECT AVG(salary) FROM employees) AS salary_difference FROM employees;

```